Pandas Logo Seaborn Logo Plotly Logo Altair Logo

Pandas **Seaborn** **Plotly** **Altair**

Introduction aux Diagrammes à Barres

Les diagrammes à barres sont un outil de visualisation fondamental utilisé pour afficher des données catégorielles avec des barres rectangulaires. La longueur de chaque barre est proportionnelle à la valeur qu'elle représente. Les diagrammes à barres sont excellents pour comparer des quantités entre différentes catégories.

Quand utiliser les diagrammes à barres :

- Comparaison de données catégorielles.
- Affichage de la fréquence ou du nombre de différentes catégories.
- Visualisation des tendances entre les catégories.

Terminologie de base :

- Barres : Les colonnes rectangulaires représentant les valeurs des données.
- Axes : Les lignes horizontales (axe des x) et verticales (axe des y) qui définissent le graphique.
- Étiquettes : Le texte qui décrit les catégories et les valeurs.
- Titre : La description générale du graphique.

Exercice : Analyse de Diamants avec les Bibliothèques de Visualisation

Avant de commencer cet exercice, veuillez lire la documentation des bibliothèques suivantes pour vous familiariser avec leurs fonctionnalités de base :

- Matplotlib : https://matplotlib.org/
- Seaborn : https://seaborn.pydata.org/
- Plotly : https://plotly.com/python/
- Altair : https://altair-viz.github.io/

Vous disposez d'un ensemble de données contenant des informations sur divers diamants. Votre tâche est de réaliser l'analyse et les visualisations suivantes en utilisant

Matplotlib, Seaborn, Plotly et Altair. Pour chaque visualisation, fournissez un bref commentaire sur ce que vous observez concernant les forces et les faiblesses de chaque bibliothèque pour cette tâche particulière.

1. **Chargement et Inspection des Données :**
   - Chargez l'ensemble de données à partir de l'URL fourni : `"https://raw.githubusercontent.com/TrainingByPackt/Interactive-Data-Visualization-with-Python/master/datasets/diamonds.csv"`
   - Affichez les 5 premières lignes de l'ensemble de données pour comprendre sa structure.
   - Imprimez les noms des colonnes et leurs types de données.
   - Affichez les 10 premières lignes de l'ensemble de données.
2. **Visualisation par Diagramme à Barres :**
   - **Matplotlib :**
     - Créez un diagramme à barres pour visualiser le nombre de diamants pour chaque catégorie de 'cut' en utilisant Matplotlib.
     - Personnalisez ce diagramme à barres en ajoutant des étiquettes appropriées pour l'axe des x et l'axe des y, et définissez un titre pour le graphique.
     - Commentez la syntaxe de Matplotlib, ses capacités de personnalisation et sa facilité d'utilisation pour cette tâche.
   - **Seaborn :**
     - Créez le même diagramme à barres (nombre de diamants par 'cut') en utilisant Seaborn.
     - Personnalisez le graphique comme vous l'avez fait avec Matplotlib (étiquettes, titre).
     - Comparez le code de Seaborn à celui de Matplotlib. Qu'est-ce que Seaborn fait différemment ou plus facilement ? Comment le style par défaut se compare-t-il ?
   - **Plotly :**
     - Créez un diagramme à barres interactif (nombre de diamants par 'cut') en utilisant Plotly.
     - Personnalisez le graphique avec des étiquettes et un titre. Explorez les fonctionnalités d'interactivité de Plotly (info-bulles, zoom).
     - Commentez l'interactivité de Plotly, sa syntaxe et en quoi elle diffère de Matplotlib et Seaborn.
   - **Altair :**
     - Créez un diagramme à barres (nombre de diamants par 'cut') en utilisant Altair.
     - Personnalisez le graphique avec des étiquettes et un titre.

    - Commentez la syntaxe déclarative d'Altair et comment elle se compare aux autres bibliothèques.

3. **Analyse Supplémentaire (Facultatif) :**

    - Explorez davantage les données en créant des diagrammes à barres pour d'autres variables catégorielles telles que 'color' ou 'clarity' dans n'importe laquelle des bibliothèques.
    - Essayez de visualiser le prix moyen pour chaque 'cut' ou 'color' en utilisant les différentes bibliothèques et commentez la façon dont chaque bibliothèque gère ce type d'agrégation.

Conseils :

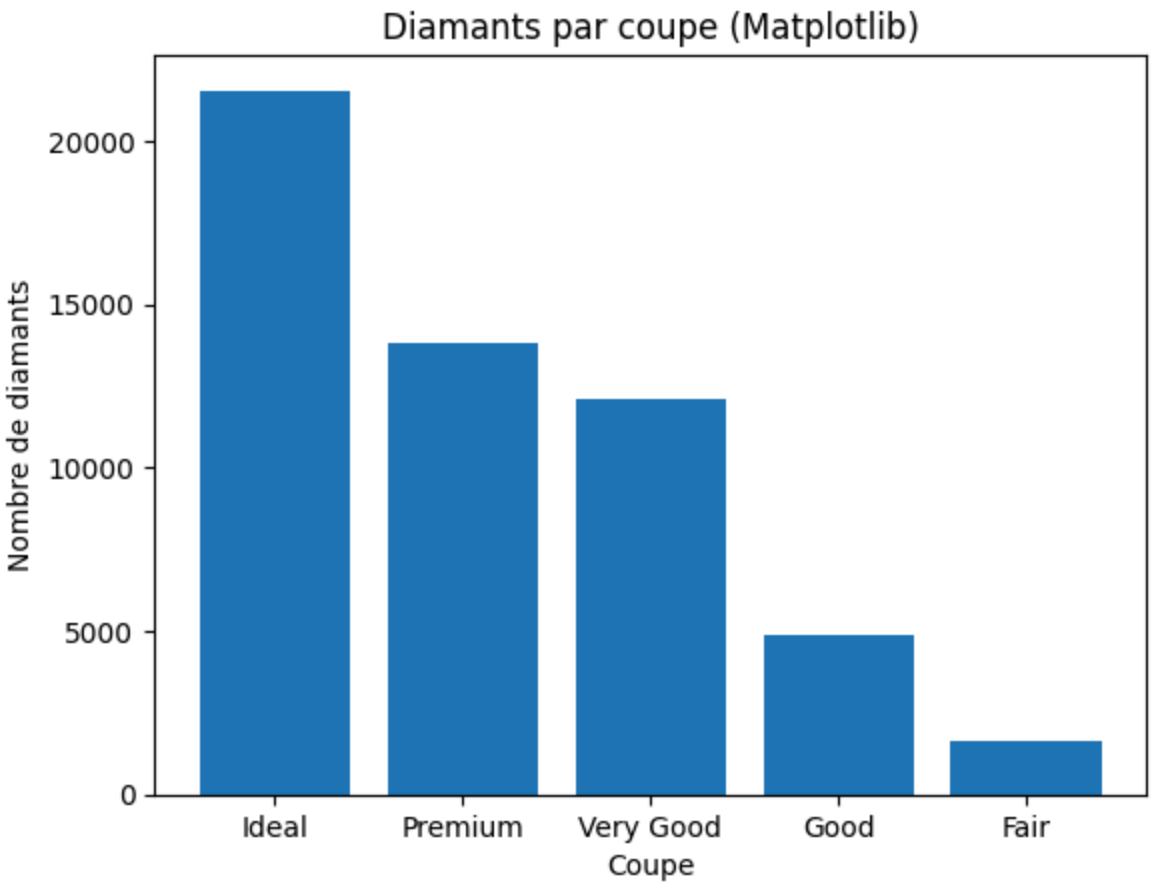
- Utilisez la bibliothèque pandas pour charger et manipuler les données.
- Utilisez les bibliothèques Matplotlib, Seaborn, Plotly et Altair pour créer les diagrammes à barres.
- Référez-vous au notebook pour des exemples de code et des explications sur les fonctions dont vous pourriez avoir besoin.

In [10]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import altair as alt

url = "https://raw.githubusercontent.com/TrainingByPackt/Interactive-Data-Vi
df = pd.read_csv(url)

print(df.head())
print(df.dtypes)
print(df.head(10))
```

```
   carat      cut color clarity  depth  table  price     x     y     z
0   0.23    Ideal     E     SI2   61.5   55.0    326  3.95  3.98  2.43
1   0.21  Premium     E     SI1   59.8   61.0    326  3.89  3.84  2.31
2   0.23     Good     E     VS1   56.9   65.0    327  4.05  4.07  2.31
3   0.29  Premium     I     VS2   62.4   58.0    334  4.20  4.23  2.63
4   0.31     Good     J     SI2   63.3   58.0    335  4.34  4.35  2.75
carat      float64
cut            str
color          str
clarity        str
depth      float64
table      float64
price        int64
x          float64
y          float64
z          float64
dtype: object
   carat        cut color clarity  depth  table  price     x     y     z
0   0.23      Ideal     E     SI2   61.5   55.0    326  3.95  3.98  2.43
1   0.21    Premium     E     SI1   59.8   61.0    326  3.89  3.84  2.31
2   0.23       Good     E     VS1   56.9   65.0    327  4.05  4.07  2.31
3   0.29    Premium     I     VS2   62.4   58.0    334  4.20  4.23  2.63
4   0.31       Good     J     SI2   63.3   58.0    335  4.34  4.35  2.75
5   0.24  Very Good     J    VVS2   62.8   57.0    336  3.94  3.96  2.48
6   0.24  Very Good     I    VVS1   62.3   57.0    336  3.95  3.98  2.47
7   0.26  Very Good     H     SI1   61.9   55.0    337  4.07  4.11  2.53
8   0.22       Fair     E     VS2   65.1   61.0    337  3.87  3.78  2.49
9   0.23  Very Good     H     VS1   59.4   61.0    338  4.00  4.05  2.39
```

2a. Matplotlib — Diagramme à barres (nombre de diamants par coupe)

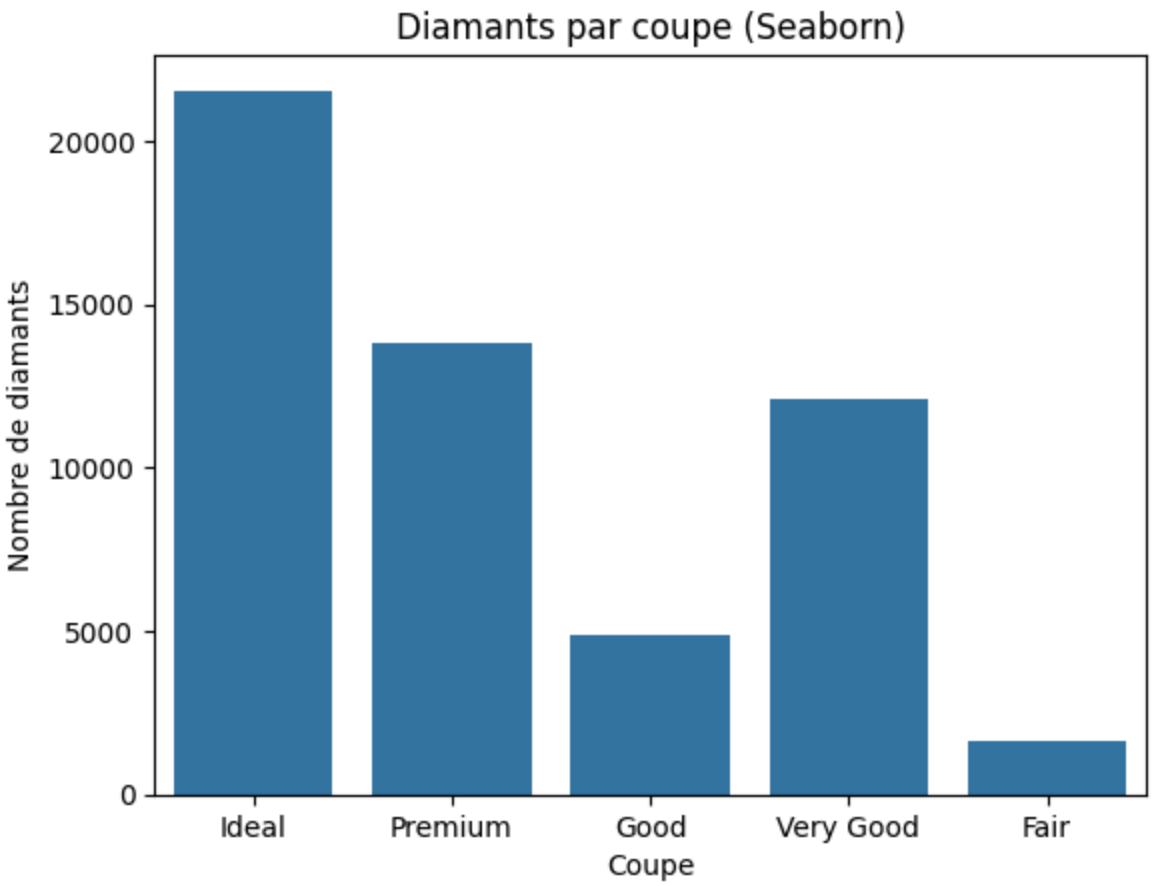
In [11]:
```
cut_counts = df['cut'].value_counts()
plt.bar(cut_counts.index, cut_counts.values)
plt.xlabel("Coupe")
plt.ylabel("Nombre de diamants")
plt.title("Diamants par coupe (Matplotlib)")
plt.show()
```



Matplotlib : Faut tout écrire à la main, c'est un peu long mais ça marche bien. On contrôle tout mais c'est pas le plus rapide.

2b. Seaborn — Diagramme à barres (nombre de diamants par coupe)

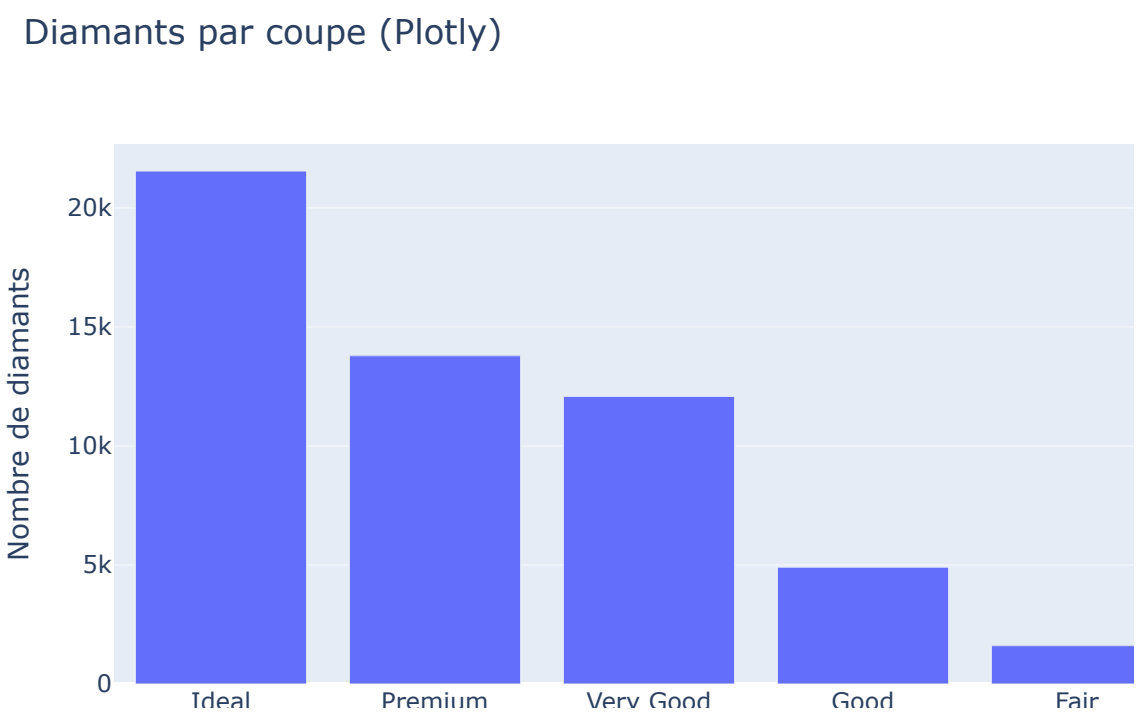
In [12]:
```
sns.countplot(data=df, x='cut')
plt.xlabel("Coupe")
plt.ylabel("Nombre de diamants")
plt.title("Diamants par coupe (Seaborn)")
plt.show()
```



Seaborn : Beaucoup plus simple, `countplot` fait le compte tout seul. Le rendu est plus joli par défaut que Matplotlib avec moins de code.

2c. Plotly — Diagramme à barres interactif (nombre de diamants par coupe)

In [13]:
```
cut_counts = df['cut'].value_counts().reset_index()
cut_counts.columns = ['cut', 'count']
fig = px.bar(cut_counts, x='cut', y='count', title="Diamants par coupe (Plot
             labels={'cut': 'Coupe', 'count': 'Nombre de diamants'})
fig.show()
```

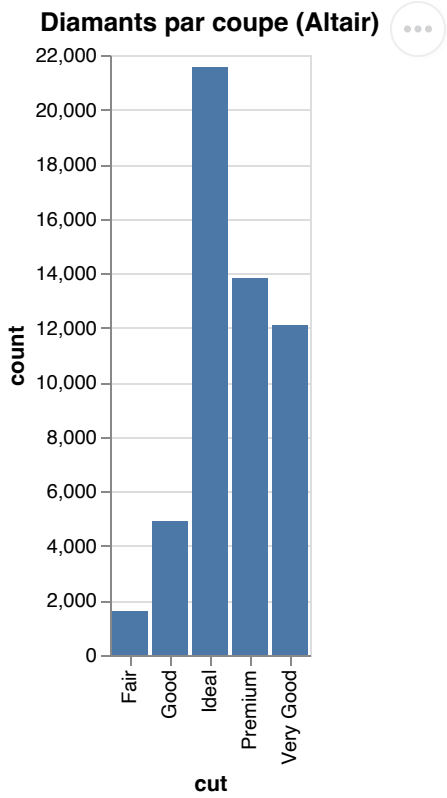


Plotly : Le graphique est interactif, on peut zoomer et voir les valeurs en passant la souris. C'est le plus pratique pour explorer les données.

2d. Altair — Diagramme à barres (nombre de diamants par coupe)

In [14]:
```
cut_counts = df['cut'].value_counts().reset_index()
cut_counts.columns = ['cut', 'count']

alt.Chart(cut_counts).mark_bar().encode(
    x='cut:N',
    y='count:Q'
).properties(title="Diamants par coupe (Altair)")
```

Out[14]:

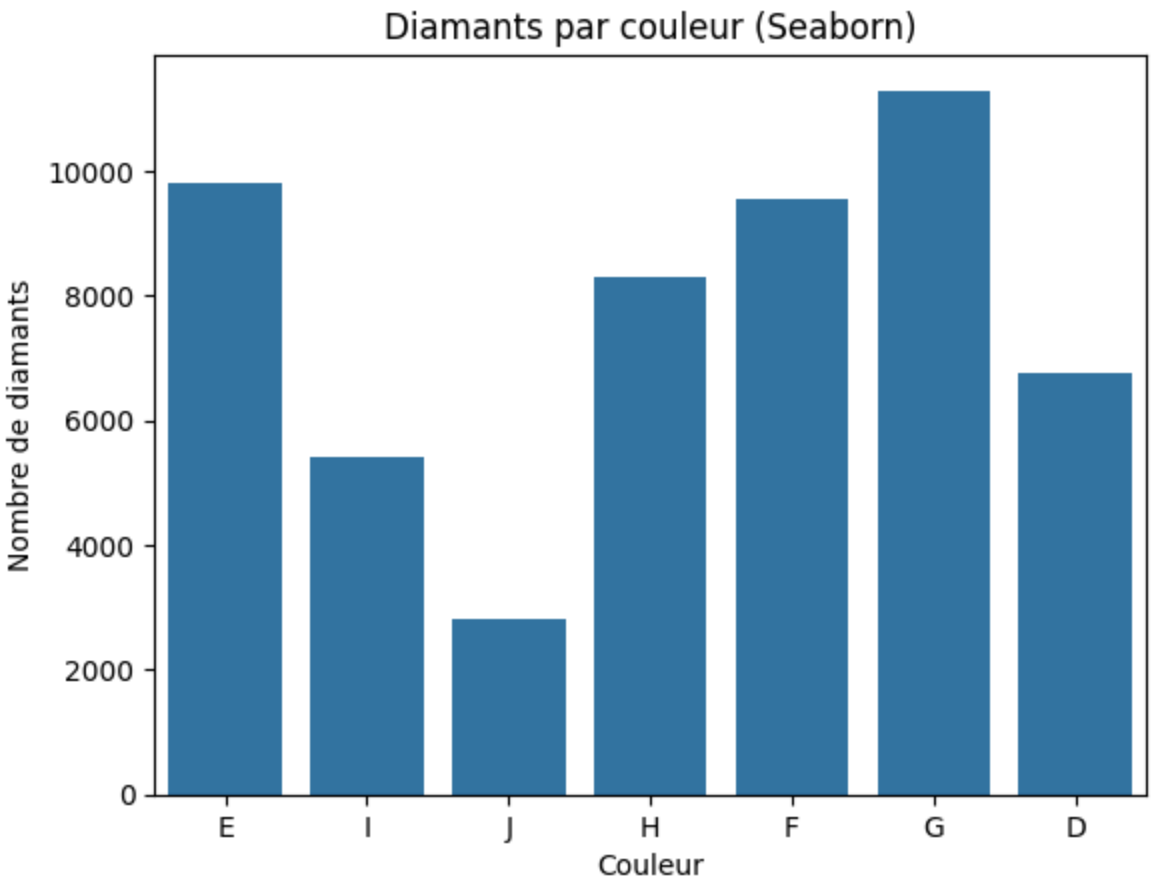


Altair : La syntaxe est différente, on décrit ce qu'on veut afficher plutôt que comment le dessiner. C'est lisible mais faut s'y habituer. Attention, il supporte pas les gros datasets directement.

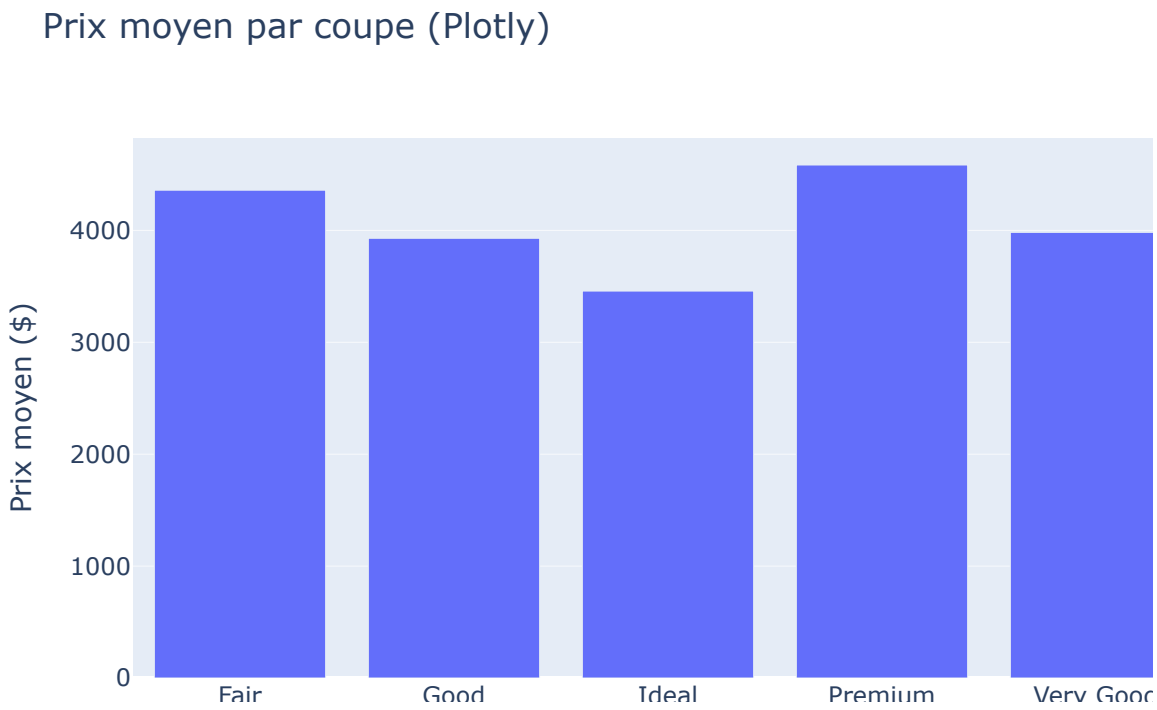
3. Analyse Supplémentaire (Facultatif)

In [19]:

```
sns.countplot(data=df, x='color')
plt.xlabel("Couleur")
plt.ylabel("Nombre de diamants")
plt.title("Diamants par couleur (Seaborn)")
plt.show()
```

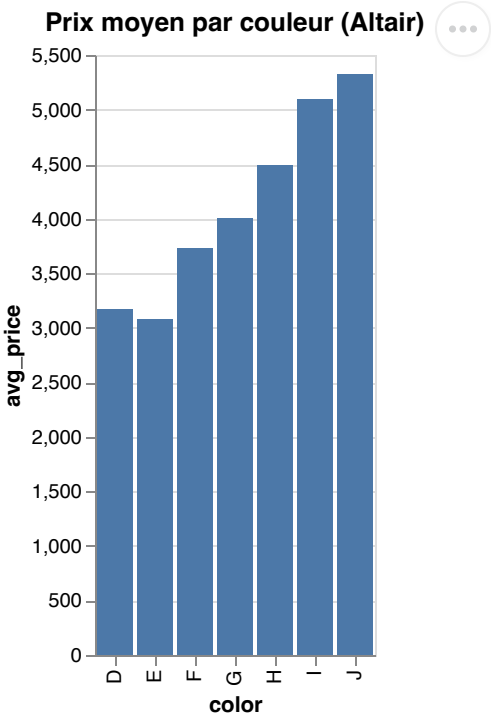


In [20]:
```
avg_cut = df.groupby('cut')['price'].mean().reset_index()
avg_cut.columns = ['cut', 'avg_price']

fig3 = px.bar(avg_cut, x='cut', y='avg_price',
              title="Prix moyen par coupe (Plotly)",
              labels={'cut': 'Coupe', 'avg_price': 'Prix moyen ($)'})
fig3.show()
```



In [18]:
```
avg_color = df.groupby('color')['price'].mean().reset_index()
avg_color.columns = ['color', 'avg_price']

alt.Chart(avg_color).mark_bar().encode(
    x='color:N',
    y='avg_price:Q',
    tooltip=['color:N', 'avg_price:Q']
).properties(title="Prix moyen par couleur (Altair)")
```

Out[18]:


Prix moyen par couleur (Altair)
5,500
5,000
4,500
4,000
3,500
3,000
2,500
2,000
1,500
1,000
500
0
avg_price
D
E
F
G
H
I
J
color